Cybersecurity MSc: Applied Cryptography 2025-26

1st Project: Extending the OpenSSL command-line tool with new block and stream ciphers through an OpenSSL 3 provider

September 28, 2026 Due date: November 8, 2026

Changelog

- v1.0 - Initial version.

1 Introduction

The `openssl` command-line tool is a powerful tool to perform isolated cryptographic operations. This tool is extensible in two ways: by changing its source code or by using external providers. In the project we will explore the second possibility.

1.1 Project goal

In this project, students will develop an OpenSSL 3 provider that adds two symmetric encryption algorithms that never have been part of it:

1. One block cipher;
2. One stream cipher.

The provider must expose both algorithms through the OpenSSL 3 EVP provider interface, so that the standard `openssl` command-line tool can use them for:

- Listing available algorithms;
- Speed benchmarking;
- Whole-file encryption;
- Whole-file decryption.

The expected result is not a modified `openssl` executable. Instead, students should build a separate provider module that can be loaded dynamically by OpenSSL.

Relevant OpenSSL documentation:

- OpenSSL provider overview: `https://docs.openssl.org/3.0/man7/provider/`
- OpenSSL provider base interface: `https://docs.openssl.org/3.0/man7/provider-base/`
- OpenSSL provider cipher interface: `https://docs.openssl.org/3.0/man7/provider-cipher/`
- `openssl list`: `https://docs.openssl.org/3.0/man1/openssl-list/`
- `openssl speed`: `https://docs.openssl.org/3.0/man1/openssl-speed/`
- `openssl enc`: `https://docs.openssl.org/3.0/man1/openssl-enc/`

1.2 Learning objectives

By the end of the project, students should be able to:

- Explain the OpenSSL 3 provider architecture;
- Distinguish between OpenSSL's command-line tool, libcrypto, EVP, and providers;
- Implement provider-side EVP cipher functions;
- Expose new cipher names to OpenSSL;
- Load a custom provider from the command line;
- Use `openssl speed` to benchmark custom provider algorithms;
- Use `openssl enc` to encrypt and decrypt files with provider-backed ciphers;
- Validate cipher implementations using known-answer tests.

2 Homework

Each group must implement:

Requirement	Description
Block cipher	One block cipher that was never part of OpenSSL
Stream cipher	One stream cipher that was never part of OpenSSL
OpenSSL provider	A dynamically loadable OpenSSL 3 provider
EVP cipher support	Both algorithms must be exposed through `OSSL_OP_CIPHER`
CLI integration	Algorithms must be usable from the standard `openssl` tool
Benchmarking	Algorithms must work with `openssl speed -evp`
File encryption	Algorithms must encrypt whole files using `openssl enc`
File decryption	Algorithms must decrypt whole files using `openssl enc`
Validation	Students must provide test vectors and automated checks
Documentation	Students must document design, build, usage, tests, and limitations

Students must not modify the OpenSSL command-line tool.

You must verify that your chosen ciphers were not included in standard OpenSSL versions relevant to the project, nor in past versions. This should be documented.

A suitable verification process is:

```
$ openssl list -cipher-algorithms
```

and also checking OpenSSL documentation and source history when needed. Deprecated ciphers, or ciphers for which severe weaknesses were found, should not be addressed.

Also, you cannot use for your ciphers third-party engines, forks, patches, or other libraries implementing them for `openssl`.

2.1 Block cipher requirements

You must select a block cipher that:

- Was published or specified clearly enough to implement;
- Has a well-defined block size;
- Has a well-defined key schedules;
- Has available test vectors, or allows students to generate and justify their own.

Examples of possible candidates include, subject to verification:

Candidate	Notes	Block size bits)	Key size (bits)
HIGHT	block cipher	64	128
LEA	block cipher	128	128, 192, 256
PRESENT	block cipher	64	80, 128
PRINCE	block cipher	64	128
SIMON variant	block cipher family	32, 48, 64, 96, 128	64, 72, 96, 128, 144, 192, 256
SPECK variant	block cipher family	32, 48, 64, 96, 128	64, 72, 96, 128, 144, 192, 256
GIFT variant	block cipher familty	64	80, 128
RECTANGLE	block cipher	64	80
KATAN or KTANTAN variant	block cipher family	32, 48, 64	80

2.2 Stream cipher requirement

Students must select a stream cipher that:

- Was published or specified clearly enough to implement;
- Has well-defined key and IV or nonce handling;
- Has available test vectors, or allows students to generate and justify their own;

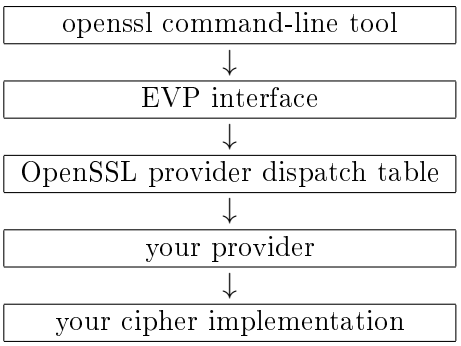
Examples of possible candidates include, subject to verification:

Candidate	Notes	Key size (bits)	IV size (bits)
Grain-128a	stream cipher	128	N/A
HC-128	eSTREAM portfolio cipher	256	256
Rabbit	eSTREAM portfolio cipher	128	64
MICKEY 2.0	eSTREAM portfolio cipher	80	0-80
Trivium	eSTREAM portfolio cipher	80	80
Salsa20 variant	eSTREAM portfolio cipher	128, 256	128
Sosemanuk	eSTREAM portfolio cipher	128-256	128
SNOW 3G	3GPP stream cipher	128, 256	128

2.3 OpenSSL integration model

You should implement the algorithms as OpenSSL 3 provider ciphers.

The architecture should be:



The provider must expose each algorithm as an `OSSL_OP_CIPHER`.

The algorithm names are up to the students, but they must be consistent, documented, and usable from the command line.

At minimum, each cipher implementation should support the following provider-side functions:

- `newctx`
- `dupctx`

- `freectx`
- `encrypt_init`
- `decrypt_init`
- `update`
- `final`
- `get_params`
- `get_ctx_params`
- `set_ctx_params`
- `gettable_params`
- `gettable_ctx_params`
- `settable_ctx_params`

The provider must correctly report:

Parameter	Block cipher example	Stream cipher example
Key length	16 bytes	16 bytes
IV length	8 or 16 bytes, depending on mode	cipher-specific nonce or IV size
Block size	cipher block size, or 1 for stream-like modes	1
Mode	ECB, CBC, CTR-like, other	stream
Padding support	optional, but recommended for CBC	N/A
Operation type	encryption and decryption	encryption and decryption

While implementing your ciphers you should consider the following relevant specifications:

- Key size: exact number of bytes accepted;
- IV or nonce size: exact number of bytes accepted;
- Endianness: especially important for counters;
- Counter increment rule: for CTR-like modes;
- Padding: whether supported;
- Decryption: whether symmetric with encryption;

2.4 Block cipher mode requirement

A raw block cipher is usually not enough for whole-file encryption. Therefore, you must expose the block cipher in at least one usable mode. Acceptable options are:

- Mandatory: ECB, CBC, CTR;
- Optional: OFB, CFB.

For ECB and CBC, you must handle padding (unless users pass `-nopad`).

2.5 File encryption requirement

Students must demonstrate file encryption and decryption using the unmodified `openssl` command. This is an example for encrypting with the PRESENT block cipher in CTR mode:

```
$ openssl enc -provider-path ./provider -provider customprov -PRESENT-128-CTR
-K 00112233445566778899aabbccddeeff -iv 0102030405060708 -in plaintext.bin -out
ciphertext.bin
```

and for decrypting:

```
$ openssl enc -d -provider-path ./provider -provider customprov -PRESENT-128-CTR
-K 00112233445566778899aabbccddeeff -iv 0102030405060708 -in ciphertext.bin -out
decrypted.bin
```

For a stream cipher, such as TRIVIUM:

```
$ openssl enc -provider-path ./provider -provider customprov -TRIVIUM -K
00112233445566778899aabbccddeeff -iv 0102030405060708090a -in plaintext.bin -out
ciphertext.bin
```

and:

```
$ openssl enc -d -provider-path ./provider -provider customprov -TRIVIUM -K
00112233445566778899aabbccddeeff -iv 0102030405060708090a -in ciphertext.bin -out
decrypted.bin
```

The actual algorithm names, key sizes, and IV sizes must match the student implementation.

In both cases, you should verify the success of the encryption-decryption cycle:

```
$ cmp plaintext.bin decrypted.bin
```

2.6 Speed evaluation requirement

You must benchmark your ciphers using `openssl speed`. This is an example for the PRESENT block cipher:

```
$ openssl speed -provider-path ./provider -provider customprov -evp PRESENT-128-CTR
```

and another for the TRIVIUM stream cipher:

```
$ openssl speed -provider-path ./provider -provider customprov -evp TRIVIUM
```

You should compare your results with at least two built-in OpenSSL ciphers, for example:

- AES-128-CBC
- AES-128-CTR
- ChaCha20

The comparison does not need to be favourable. The goal is to understand provider overhead, implementation efficiency, and algorithm behaviour.

The final report should include a table similar to:

Cipher	Mode	16 bytes	64 bytes	256 bytes	1024 bytes	8192 bytes
Student block cipher	ECB	···	···	···	···	···
AES-128-ECB	ECB	···	···	···	···	···
Student block cipher	CBC	···	···	···	···	···
AES-128-CBC	CBC	···	···	···	···	···
Student block cipher	CTR	···	···	···	···	···
AES-128-CTR	CTR	···	···	···	···	···
Student stream cipher	stream	···	···	···	···	···
ChaCha20	stream	···	···	···	···	···

You must specify:

- OpenSSL version;
- Compiler;
- Compiler flags;

- CPU model;
- Operating system;
- Whether hardware acceleration was enabled;
- Whether the provider was built in debug or release mode.

2.7 Validation requirements

You must provide evidence that both ciphers work correctly. At minimum, you must attend to these requirements:

1. Known-answer test for the block cipher core.
2. Known-answer test for each block cipher mode.
3. Known-answer test for the stream cipher keystream.
4. File encryption and decryption round-trip test.
5. Different input sizes, including:
   - Empty file;
   - 1 byte;
   - One block;
   - One block plus 1 byte;
   - Many blocks;
   - Large file.

2.8 Suggested plan

We suggest that you follow the following phases:

1. Select algorithms, collect suitable references to algorithm specifications, get evidence that the algorithms were not part of standard OpenSSL, select operational details (key sizes, IV sizes, block sizes, expected modes);
2. Implement standalone ciphers and test them with unitary tests and known-answer tests;
3. Implement provider skeleton (initialization function, dispatch table, name, version and parameters, successful listing with `openssl`;
4. Expose a block cipher through EVP (and successfully listing it with `openssl`, encrypt/decrypt files and perform speed evaluations);
5. Expose a stream cipher through EVP (and successfully listing it with `openssl`, encrypt/decrypt files and perform speed evaluations);
6. Benchmark and report your ciphers (benchmark scripts and tables, discussion of results, final report).

3 Homework delivery

Send your code to the course teachers through eLearning (a submission link will be provided). Include a small report, with no more than 20 pages, describing the implementation (not a complete copy of the code developed!). Code snippets may be used to illustrate your implementation.

Every piece of code imported from anywhere must be stated in the report and in the code itself. Failure to do so will be penalised.

3.1 Grading

The following grading components will be used:

- Standalone block cipher implementation (15%);
- Standalone stream cipher implementation (15%);
- OpenSSL provider architecture (20%);
- EVP integration and CLI usability (15%);
- Testing and validation (15%);
- Benchmarking (15%);
- Documentation and report quality | 5%);

A stronger submission must:

- Have clean separation between cipher code and provider glue;
- Include reproducible build instructions;
- Include automated tests;
- Include known-answer test vectors;
- Support arbitrary file sizes;
- Provide meaningful benchmark comparison;
- Clearly document limitations.